



第10章

线性搜索与排序算法

教师：肖依永 | 刘杰 | 周晟瀚
可靠性与系统工程学院

1

线性搜索算法

2

排序算法

3

算法的复杂度

1 线性搜索算法

line search

什么是线性搜索？

在许多迭代法中，有一个共同特点，就是得到 $\mathbf{x}^{(k)}$ 后，需要按照某种方法确定一个下降方向 $\mathbf{d}^{(k)}$ ，沿着该下降方向搜索一个步长 λ ，以获得更优的点 $\mathbf{x}^{(k+1)}$ 。然后重复这样的操作，直到获得最优解。

即当 $\mathbf{d}_k$ 确定后，搜索一个 α ，使下面函数最小化：

$$\min_{\alpha > 0} f(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}), \quad \mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}$$

线性搜索也称一维搜索。这种方法不仅对于解决一维最优化问题具有实际意义，而且也是求解多维最优化问题的重要支柱。

计算步长 α_k 的两种线性搜索：精确搜索和非精确搜索：

精确搜索： α_k 是一维优化问题的解

$$\min_{\alpha > 0} f(x_k + \alpha d_k)$$

即 α_k 满足

$$\nabla f(x_k + \alpha_k d_k)^T d_k = 0$$

非精确搜索： α_k 按照某种规则计算使之满足

$$f(x_k + \alpha_k d_k) < f(x_k)$$

即函数值有一定程度下降.

精确搜索： α_k 是一维优化问题的解

$$\min_{\alpha > 0} f(x_k + \alpha d_k)$$

令 $\phi(\alpha) = f(x_k + \alpha d_k)$, 则 $\phi'(\alpha) = \nabla f(x_k + \alpha d_k)^T d_k$

由 $\phi'(\alpha) = 0$ 得解 α_k . 所以计算 α_k 等价于解方程

$$\nabla f(x_k + \alpha d_k)^T d_k = 0$$

如果方程简单, 直接解方程求得 α_k ; 否则需用数值方法求近似解

直接解方程计算步长 α_k 的例子：

如二次函数

$$f(x) = \frac{1}{2} x^T Q x + q^T x + c, \quad x \in R^n$$

其中矩阵 Q 对称且正定

由于

$$\phi(\alpha) = f(x_k + \alpha d_k) = \frac{\alpha^2}{2} d_k^T Q d_k + \alpha \nabla f(x_k)^T d_k + f(x_k)$$

则
$$\phi'(\alpha) = \alpha d_k^T Q d_k + \nabla f(x_k)^T d_k$$

令 $\phi'(\alpha) = 0$, 得解

$$\alpha_k = -\frac{\nabla f(x_k)^T d_k}{d_k^T Q d_k}$$

S^k 方向上的任何一点可以表示为

$$X^{k+1} = X^k + \alpha_k S^k$$

其中 α 是步长因子，为实系数，此时 S^k 方向上任何一点的目标函数值

为 $f(X^k + \alpha_k S^k)$ ，它是参数 α 的一元函数。那么在沿 S^k 方向求 $f(X)$ 的极小点，这就是求一元函数 $f(X^k + \alpha_k S^k)$ 的极小问题，它可表示为：

$$\alpha: \min f(X^k + \alpha_k S^k)$$

这个过程称为一维搜索过程。

如： $F(X) = x_1^2 + x_2^2 - 8x_1 - 12x_2 + 52$

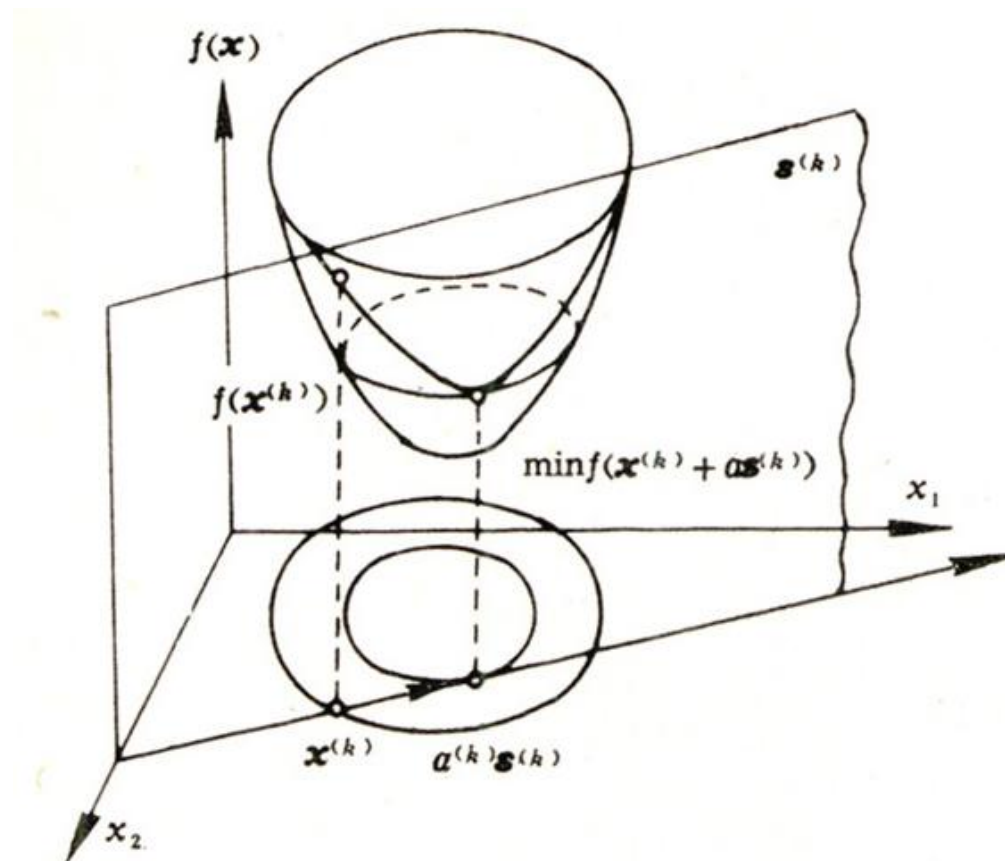
当 $X^0 = [0 \ 0]^T, d^0 = [1 \ 1]^T$

$$X = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \alpha \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha \\ \alpha \end{bmatrix}$$

则 $F = x_1^2 + x_2^2 - 8x_1 - 12x_2 + 52 = 2\alpha^2 - 20\alpha + 52$

一维搜索示意图

$$X^{k+1} = X^k + \alpha S^k (k = 0, 1, 2 \dots)$$



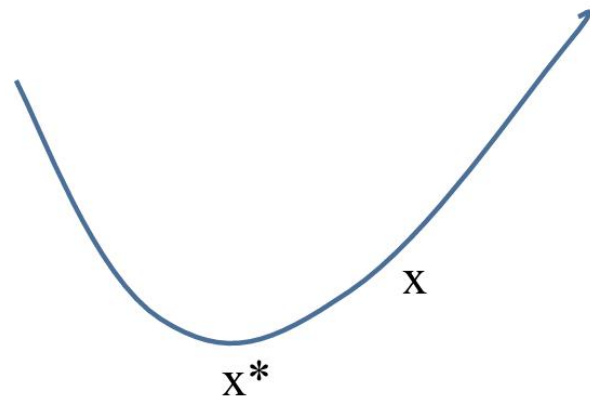
4.3 最速下降搜索法

最速下降求解法基本原理：

考虑连续可微的方程组 $\mathbf{F}(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x})]^T = \mathbf{0}$,

构造函数 $\varphi(\mathbf{x}) = \frac{1}{2} \mathbf{F}(\mathbf{x})^T \mathbf{F}(\mathbf{x}) = \frac{1}{2} [f_1(\mathbf{x})^2 + f_2(\mathbf{x})^2 + \dots + f_m(\mathbf{x})^2]$,

- (1) 求解 $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ 转化为 求解 $\min \varphi(\mathbf{x})$;
- (2) $\varphi(\mathbf{x})$ 是连续函数, 最优解 $\mathbf{x}^*$ 处具有最低值;
- (3) 根据当前解 $\mathbf{x}$, 以其负梯度方向 $-\nabla \varphi(\mathbf{x})$ 作为搜索方向, 搜索计算令 $\varphi(\mathbf{x})$ 下降最大的步长 Δh , 并以此步长移动当前解, 直到当前解无限靠近最优解 $\mathbf{x}^*$ 。



4.3 最速下降搜索法

算法步骤:

第 0 步: 设定初始解 $\mathbf{x}^{(0)} = [1, 1, 1]$, 令 $k \leftarrow 0$;

第 1 步: 计算 $F(\mathbf{x}^{(k)})$, 计算 $\varphi(\mathbf{x}^{(k)})$, 判断: 如果 $\|F(\mathbf{x}^{(k)})\|$ 满足精度, 则转第 6 步;

第 2 步: 计算 $\nabla F(\mathbf{x}^{(k)})$;

第 3 步: 计算下降方向 $\mathbf{p}(\mathbf{x}^{(k)}) = (F(\mathbf{x}^{(k)}))^T \nabla F(\mathbf{x}^{(k)})$;

第 4 步: 计算步长 $a = \varphi(\mathbf{x}^{(k)}) / (\mathbf{p}(\mathbf{x}^{(k)})^T \mathbf{p}(\mathbf{x}^{(k)}))$;

第 5 步: 令 $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - a\mathbf{p}(\mathbf{x}^{(k)})$, $k \leftarrow k + 1$; 转第 1 步。

第 6 步: 计算结束, 输出结果。

(2) 确定近似最优搜索步长 α_k

$$\begin{aligned}\varphi(\mathbf{x}^{(k+1)}) &= \varphi(\mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}) \\ &\approx \varphi(\mathbf{x}^{(k)}) + (\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)})^T \nabla \varphi(\mathbf{x}^{(k)}) \\ &= \varphi(\mathbf{x}^{(k)}) + \alpha_k (\mathbf{p}^{(k)})^T \nabla \varphi(\mathbf{x}^{(k)}) \\ &= \varphi(\mathbf{x}^{(k)}) - \alpha_k (\mathbf{p}^{(k)})^T (\mathbf{p}^{(k)})\end{aligned}$$

$$\alpha_k \text{ 应使 } \varphi(\mathbf{x}^{(k+1)}) \rightarrow 0, \text{ 故 } \alpha_k = \frac{\varphi(\mathbf{x}^{(k)})}{(\mathbf{p}^{(k)})^T \mathbf{p}^{(k)}}$$

最速下降法的计算格式:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \frac{\varphi(\mathbf{x}^{(k)})}{(\mathbf{p}^{(k)})^T \mathbf{p}^{(k)}} \mathbf{p}^{(k)} \quad k = 0, 1, 2, \dots$$

2.3 牛顿迭代法

一、 牛顿迭代公式的构造

设 $f(x)$ 在其零点 x^* 附近连续可微, 已知 x_k 为的第 k 次近似值, 则

$$\begin{aligned} f(x) &= f(x_k) + f'(x_k)(x - x_k) + O((x - x_k)^2) \\ &\approx f(x_k) + f'(x_k)(x - x_k) \stackrel{\Delta}{=} P_1(x) \end{aligned}$$

取 $P_1(x) = 0$ 的根作为 x^* 的第 $k+1$ 次近似值 x_{k+1}

解得

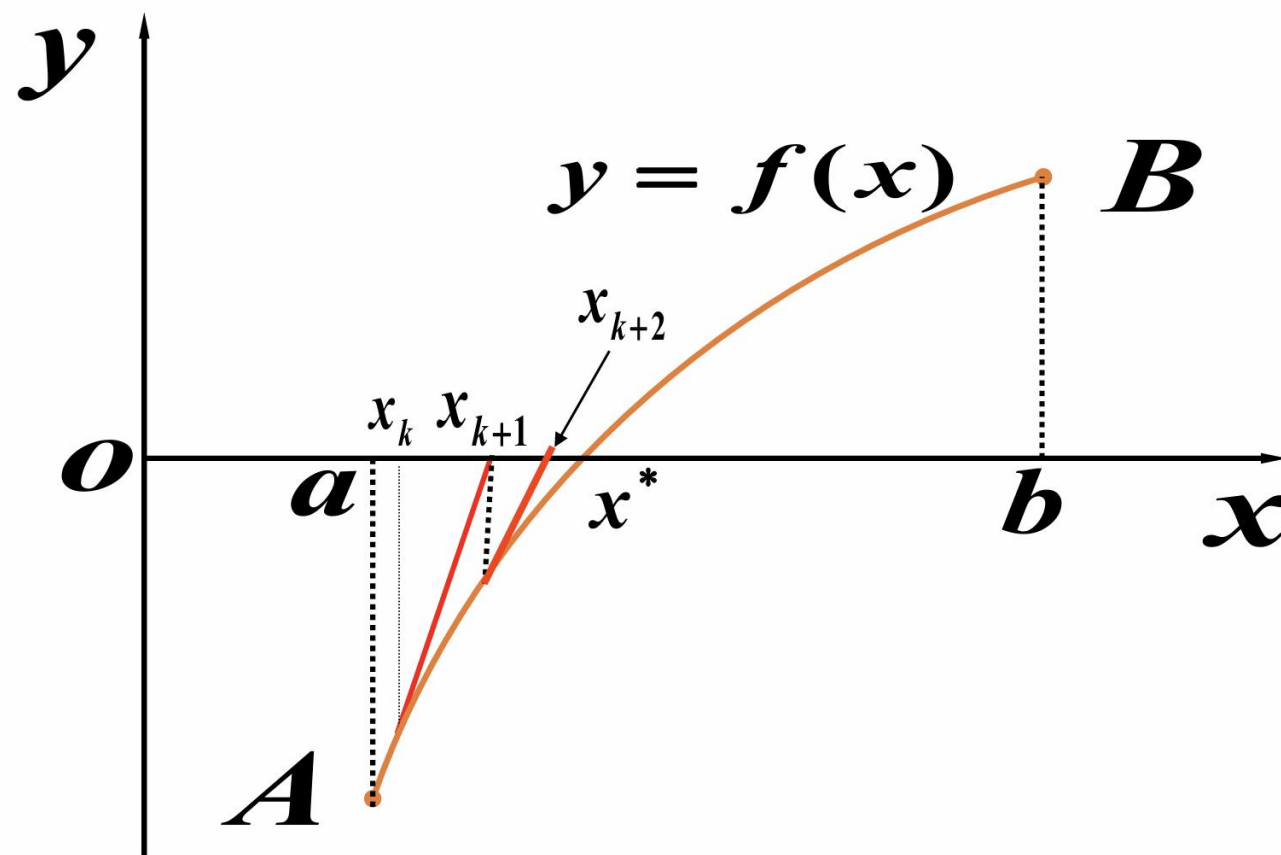
$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad (2-12)$$

牛顿迭代法

其迭代函数为

$$g(x) = x - \frac{f(x)}{f'(x)} \quad (2-13)$$

回顾-非线性方程组的数值解法



几何意义：过点 $P(x_k, f(x_k))$ 作函数 $y=f(x)$ 的切线 l ：

$$y = f(x_k) + f'(x_k)(x - x_k)$$

以切线 l 与 x 轴的交点 x_{k+1} 作为 x^* 的新近似值

求多元函数极值点，需要进行一系列的一维搜索，因此一维搜索是优化搜索方法的基础。

求解一元函数 $\varphi(\alpha)$ 的极小点 α^* ，可采用解析解法，即利用一元函数的极值条件 $\varphi'(\alpha^*) = 0$ 求 α^*

- 在用函数 $\varphi(\alpha)$ 的导数求 α^* 时，所用的函数 $\varphi(\alpha)$ 是仅以步长因子 α 为变量的一元函数，而不是以设计点 x 为变量的多元函数 $f(x)$ 。

$$f(\mathbf{x}^{k+1}) = f(\mathbf{x}^k + \alpha_k \mathbf{s}^k) = \varphi(\alpha_k)$$

解析法:

① $f(\mathbf{x}^{(k)} + \alpha \mathbf{s}^{(k)})$ 沿 $\mathbf{s}^{(k)}$ 方向在 $\mathbf{x}^{(k)}$ 点泰勒展开;

② 取二次近似:

$$f(\mathbf{x}^{(k)} + \alpha \mathbf{s}^{(k)}) \approx f(\mathbf{x}^{(k)}) + [\nabla f(\mathbf{x}^{(k)})]^T \alpha \mathbf{s}^{(k)} + \frac{1}{2} \alpha^2 [\mathbf{s}^{(k)}]^T \mathbf{G}(\mathbf{x}^{(k)}) \mathbf{s}^{(k)}$$

③ 对 α 求导, 令其为零。 $\frac{d}{d\alpha} f(\mathbf{x}^{(k)} + \alpha \mathbf{s}^{(k)}) = 0$

$$[\nabla f(\mathbf{x}^{(k)})]^T \mathbf{s}^{(k)} + \alpha [\mathbf{s}^{(k)}]^T \mathbf{G}(\mathbf{x}^{(k)}) \mathbf{s}^{(k)} = 0$$

④ 求得最优步长

$$\alpha^{(k)} = - \frac{[\nabla f(\mathbf{x}^{(k)})]^T \mathbf{s}^{(k)}}{[\mathbf{s}^{(k)}]^T \mathbf{G}(\mathbf{x}^{(k)}) \mathbf{s}^{(k)}}$$

这里是直接利用函数 $f(\mathbf{x})$ 而不需要把它化成步长因子 α 的函数 $\varphi(\alpha)$ 。不过，此时需要计算 $\mathbf{x} = \mathbf{x}^k$ 点处梯度 $\nabla f(\mathbf{x})$ 和海赛矩阵 G 。

■ 解析解法的缺点——需要进行求导计算。

- 对于函数关系复杂、求导困难或无法求导的情况，使用解析法将是非常不便的。

因此，在优化设计中，求解最佳步长因子 α^* 主要采用数值解法，利用计算机通过反复迭代计算求得最佳步长因子的近似值。

- 数值解法的基本思路是：首先确定 α^* 所在的搜索区间，然后根据区间消去法原理不断缩小此区间，从而获得的数 α^* 值近似解。

■一维问题是多维问题的基础

求目标函数 $f(X)$ 的极小点，从理论上说需要求解方程：

$$\nabla f(X) = 0 \quad \text{其中} \quad X = (x_1, x_2, \dots, x_n)^T$$

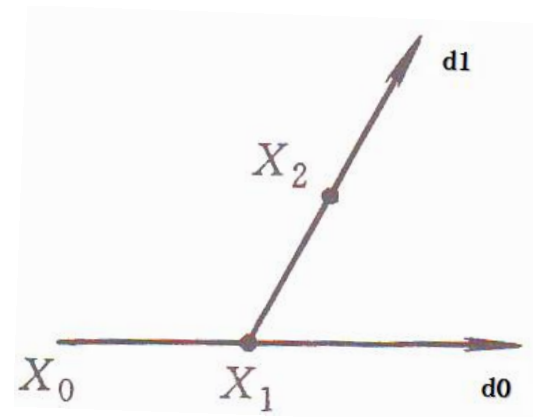
那么如何来求 $f(X)$ 的极小点呢？

基本思想：

$$X_0, X_1, \dots, X_k, X_{k+1}$$



$$f(X_0) \geq f(X_1) \geq \dots \geq f(X_k) \geq f(X_{k+1})$$



这种逐次迭代方法，在计算机上很容易实现，因此在优化设计中被广泛地采用。

■ 解析解法对于函数关系复杂、求导困难等情况难以实现。在实际优化设计中，数值解法的应用更为有效，且适合计算机的运算特点。

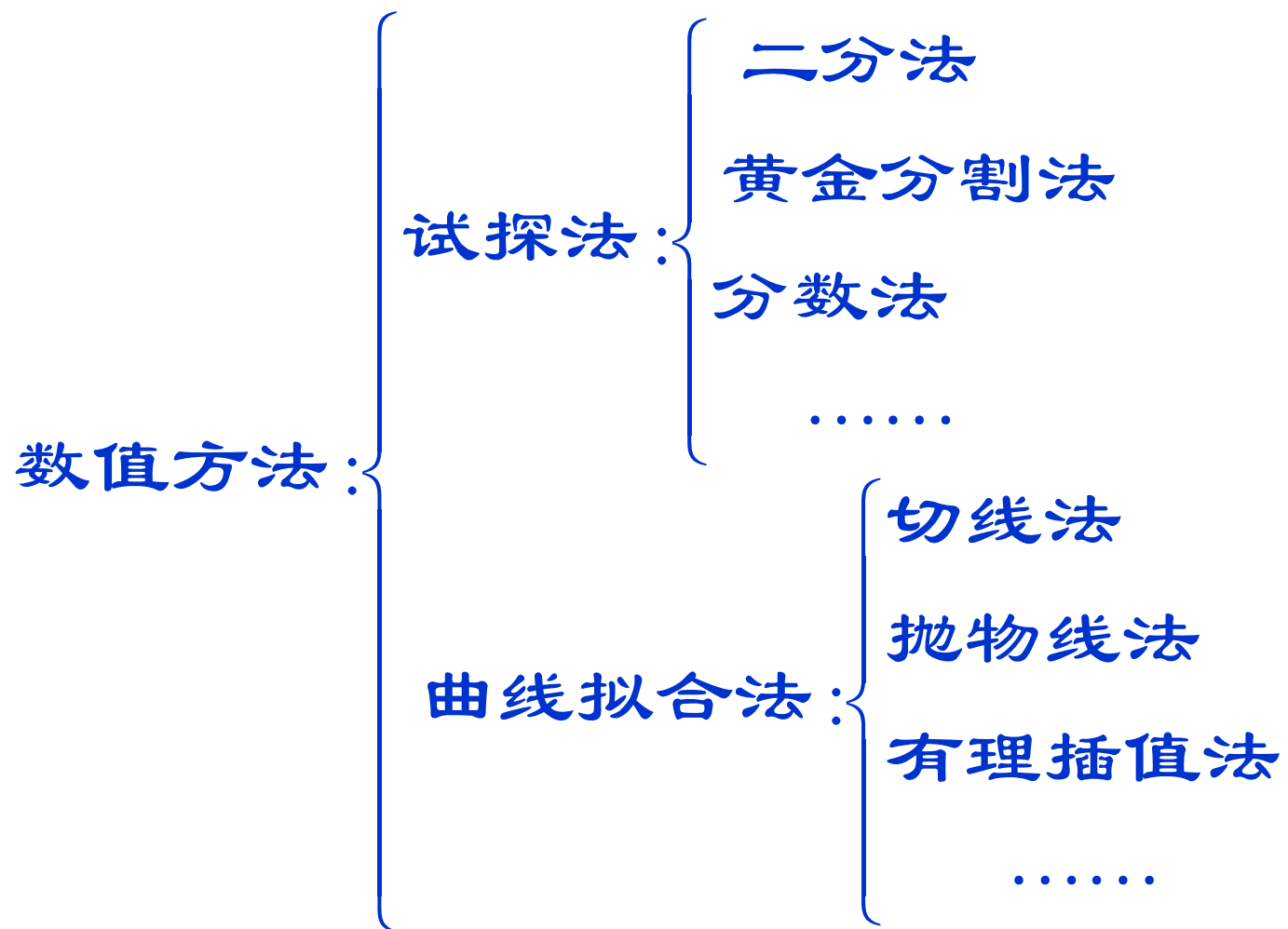
■ **数值解法**基本思路：

先确定 α_k 在的搜索区间，然后根据区间消去法原理不断缩小此区间所，从而获得 α_k 的数值近似解。

一维搜索一般分为两大步骤：

(1) 确定初始搜索区间 $[a, b]$ ，该区间应是包括一维函数极小点在内的**单谷区间**。

(2) 在单谷区间 $[a, b]$ 内通过缩小区间寻找极小点。



1.2 黄金分割法(0.618法)

- 在实际计算中，最常用的一维搜索试探方法是黄金分割法，又称作0.618法。可以通过学习黄金分割法来了解一维搜索试探方法的基本思想。
- 在搜索区间 $[a, b]$ 内适当插入两点 a_1 、 a_2 ，并计算其函数值。 a_1 、 a_2 将区间分成三段。应用函数的单谷性质，通过函数值大小的比较，删去其中一段，使搜索区间得以缩短。然后再在保留下来的区间上作同样的处置，如此迭代下去，使搜索区间无限缩小，从而得到极小点的数值近似解。

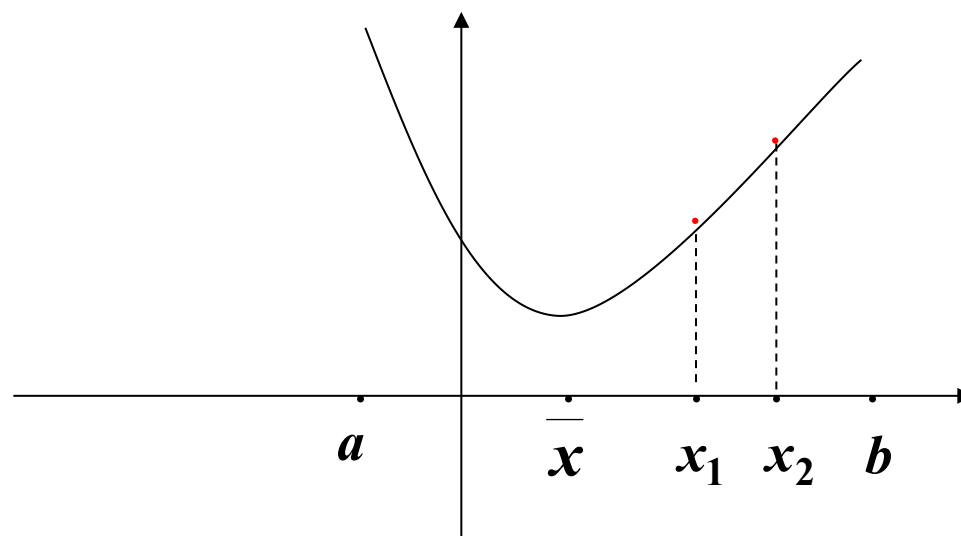
1.2 黄金分割法(0.618法)

■1. 单峰函数

定义： 设 $f(x)$ 是区间 $[a, b]$ 上的一元函数， $\bar{x}$ 是 $f(x)$ 在 $[a, b]$ 上的极小点，且对任意的 $x_1, x_2 \in [a, b], x_1 < x_2$ ，有

- (a) 当 $x_2 \leq \bar{x}$ 时， $f(x_1) > f(x_2)$;
- (b) 当 $x_1 \geq \bar{x}$ 时， $f(x_1) < f(x_2)$.

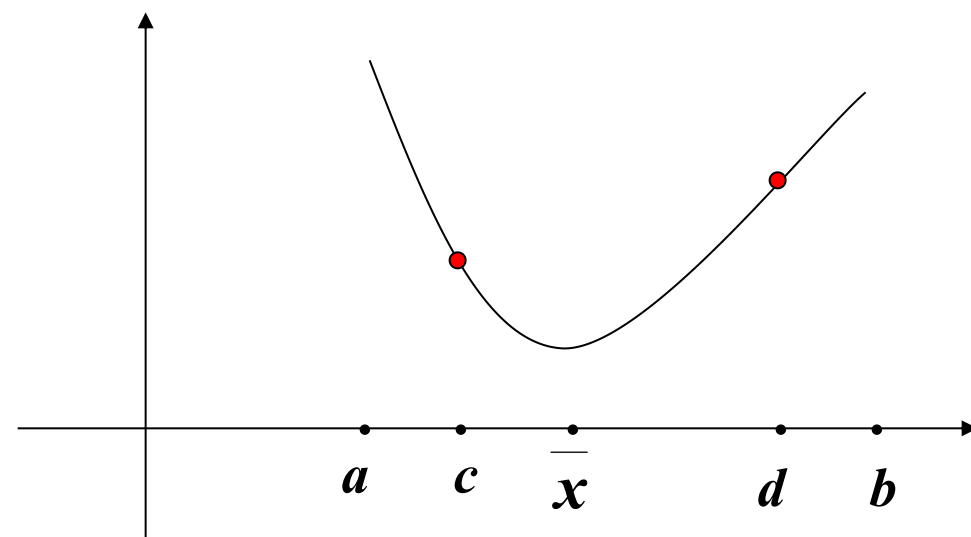
则称 $f(x)$ 是单峰函数。



性质：通过计算区间 $[a,b]$ 内两个不同点的函数值，就可以确定一个包含极小点的子区间。

定理 设 $f(x)$ 是区间 $[a,b]$ 上的单峰函数， $\bar{x}$ 是 $f(x)$ 在 $[a,b]$ 上的极小点。任取点 $c < d \in [a,b]$ ，则有

- (1) 如果 $f(c) > f(d)$ ，则 $\bar{x} \in [c, b]$;
- (2) 如果 $f(c) \leq f(d)$ ，则 $\bar{x} \in [a, d]$ 。

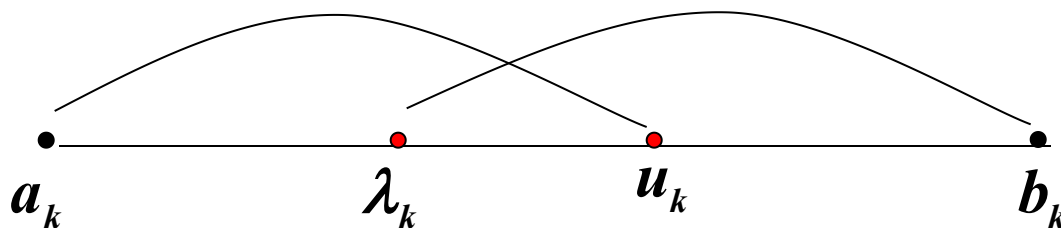


■2. 黄金分割法

思想：通过选取试探点使包含极小点的区间按相同比例不断缩短，直到区间长度小到一定程度，此时区间上各点的函数值均接近极小值。

下面推导黄金分割法的计算公式 设 $f(x)$ 在 $[a_1, b_1]$ 上单峰，极小点 $\bar{x} \in [a_1, b_1]$.

割法的计算公式



第 k 次迭代前 $\bar{x} \in [a_k, b_k]$, 取 $\lambda_k, \mu_k \in [a_k, b_k]$, 规定 $\lambda_k < \mu_k$.

计算 $f(\lambda_k)$ 和 $f(\mu_k)$, 分两种情况:

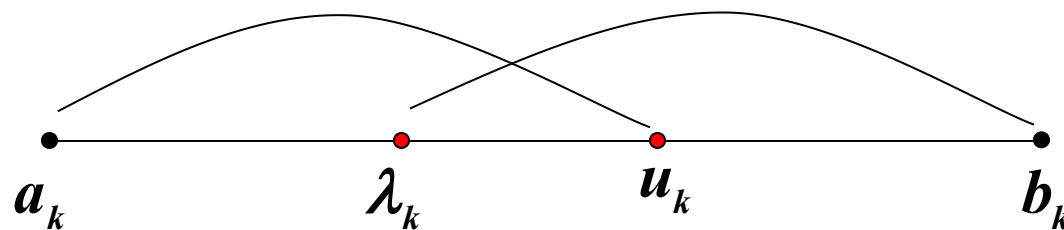
1. 若 $f(\lambda_k) > f(\mu_k)$, 则令 $a_{k+1} = \lambda_k, b_{k+1} = b_k$;

2. 若 $f(\lambda_k) \leq f(\mu_k)$, 则令 $a_{k+1} = a_k, b_{k+1} = \mu_k$.

如何确定 λ_k 与 μ_k ?

要求其满足以下两个条件：

$$1. \quad b_k - \lambda_k = \mu_k - a_k \quad (1)$$



2. 每次迭代区间长度缩短 比率相同，即

$$b_{k+1} - a_{k+1} = \alpha(b_k - a_k) \quad (\alpha > 0 \text{ 为某一缩小比例}) \quad (2)$$

由式 (1) 与 (2) 可得：

$$\begin{cases} \lambda_k = a_k + (1 - \alpha)(b_k - a_k) \end{cases} \quad (3)$$

$$\begin{cases} \mu_k = a_k + \alpha(b_k - a_k) \end{cases} \quad (4)$$

α 取值的确定 ?

通过确定 α 的取值, 使上一次迭代剩余的迭代点恰与下一次迭代的一个迭代点重合, 从而减少算法的计算量。

(1) 设在第 k 次迭代时有 $f(\lambda_k) \leq f(u_k)$, 则有

$$[a_{k+1}, b_{k+1}] = [a_k, u_k]。$$

在第 $k+1$ 次迭代时选取 λ_{k+1}, u_{k+1} , 则由 (4) 有

$$\begin{aligned} u_{k+1} &= a_{k+1} + \alpha(b_{k+1} - a_{k+1}) \\ &= a_k + \alpha^2(b_k - a_k) \end{aligned}$$

如果令 $\alpha^2 = 1 - \alpha$, 则 $u_{k+1} = \lambda_k$, 因此 u_{k+1} 不必重新计算。

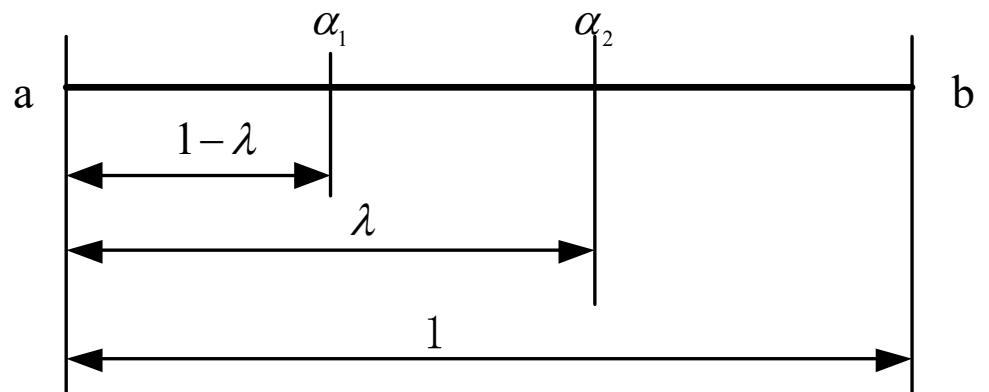
$$\alpha^2 = 1 - \alpha \Rightarrow \alpha = \frac{\sqrt{5} - 1}{2} \approx 0.618$$

(2) 若在第 k 次迭代时有 $f(\lambda_k) > f(u_k)$ 。 同理可得

算法步骤：

1. 给定初始区间 $[a_1, b_1]$, 精度要求 $\varepsilon > 0$.
令 $\lambda_1 = a_1 + 0.382(b_1 - a_1)$, $\mu_1 = a_1 + 0.618(b_1 - a_1)$,
并计算 $f(\lambda_1)$ 与 $f(\mu_1)$. 令 $k := 1$.
2. 若 $b_k - a_k < \varepsilon$, 停止, 且 $\bar{x} = \frac{b_k + a_k}{2}$. 否则,
当 $f(\lambda_k) > f(\mu_k)$ 时, 转 3; 当 $f(\lambda_k) \leq f(\mu_k)$ 时, 转 4.
3. 令 $a_{k+1} = \lambda_k$, $b_{k+1} = b_k$, $\lambda_{k+1} = \mu_k$,
 $\mu_{k+1} = a_{k+1} + 0.618(b_{k+1} - a_{k+1})$, 计算 $f(\mu_{k+1})$, 令 $k := k + 1$, 转 2。
4. 令 $a_{k+1} = a_k$, $b_{k+1} = \mu_k$, $\mu_{k+1} = \lambda_k$,
 $\lambda_{k+1} = a_{k+1} + 0.382(b_{k+1} - a_{k+1})$, 计算 $f(\lambda_{k+1})$, 令 $k := k + 1$, 转 2。

黄金分割法的迭代效果：第 k 次后迭代后所得区间长度为初始区间长度的 $(\frac{\sqrt{5}-1}{2})^k$ 倍。

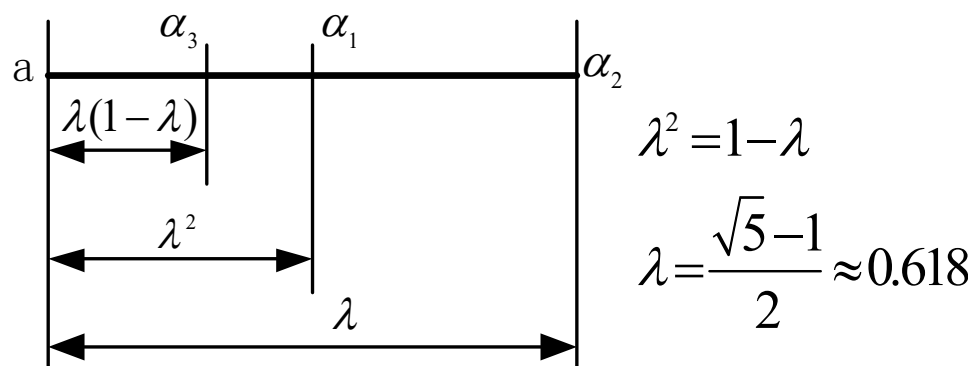


黄金分割法要求在保留下来的区间内再插入一点所形成的区间新三段，与原来区间的三段具有相同的比例分布。

两内分点值:

$$\alpha_1 = b - \lambda(b-a) = b - 0.618(b-a)$$

$$\alpha_2 = a + \lambda(b-a) = a + 0.618(b-a)$$



$$\lambda^2 = 1 - \lambda$$

$$\lambda = \frac{\sqrt{5}-1}{2} \approx 0.618$$

结论：所谓黄金分割是指将一线段分成两段的方法，使整段长与较长段的长度比值等于较长段与较短段长度的比值即 $1:\lambda = \lambda:(1-\lambda)$ 。

其它试探点算法：Fibonacci（斐波那契）算法（分数法）。

Fibonacci数列：

$$F_1 = F_2 = 1, F_{k+1} = F_k + F_{k-1}, \quad k = 1, 2, \dots$$

Fibonacci算法计算试探点的公式为：

给定计算次数 $n, b_n - a_n \leq \varepsilon$

$$\lambda_k = a_k + \frac{F_{n-k-1}}{F_{n-k+1}}(b_k - a_k), \quad k = 1, 2, \dots, n-1 \quad b_{k+1} - a_{k+1} = \frac{F_{n-k}}{F_{n-k+1}}(b_k - a_k)$$

$$\mu_k = a_k + \frac{F_{n-k}}{F_{n-k+1}}(b_k - a_k), \quad k = 1, 2, \dots, n-1 \quad \text{缩小比例：} \alpha_k = \frac{F_{n-k}}{F_{n-k+1}}$$

关于 n 的计算:

由递推关系可得

$$\begin{aligned} b_n - a_n &= \frac{F_{n-1}}{F_n} (b_{n-1} - a_{n-1}) = \frac{F_1}{F_2} \cdot \frac{F_2}{F_3} \cdots \frac{F_{n-1}}{F_n} (b_1 - a_1) \\ &= \frac{F_1}{F_n} (b_1 - a_1) \end{aligned}$$

故有 $\frac{F_1}{F_n} (b_1 - a_1) \leq \varepsilon$

从而 $F_n \geq \frac{b_1 - a_1}{\varepsilon}$ 由此计算出 F_n 而后得到 n

Fibonacci法为一维极小化最优策略，而黄金分割法为近似最优，但后者简单，因而较常用

Fibonacci法是黄金分割法的推广，黄金分割法是Fibonacci法的极限

Fibonacci数列递推关系的特征方程为

$$\tau^2 - \tau - 1 = 0$$

两个特征根为

$$\tau_1 = \frac{1 + \sqrt{5}}{2}, \quad \tau_2 = \frac{1 - \sqrt{5}}{2}$$

Fibonacci数列递推关系的解是

$$F_k = c_1 \tau_1^k + c_2 \tau_2^k$$

由初始条件 $F_0 = F_1 = 1$ 代入后得

$$c_1 = \frac{1 + \sqrt{5}}{2\sqrt{5}}, \quad c_2 = -\frac{1 - \sqrt{5}}{2\sqrt{5}}$$

所以

$$F_k = \frac{1}{\sqrt{5}} \left\{ \left(\frac{1 + \sqrt{5}}{2} \right)^{k+1} - \left(\frac{1 - \sqrt{5}}{2} \right)^{k+1} \right\}$$

于是

$$\lim_{n \rightarrow \infty} \frac{F_{n-1}}{F_n} = \frac{1}{\tau_1} \approx 0.618$$

当 $n \geq 7$ 时, $\frac{F_{n-1}}{F_n} \approx 0.618$ (三位有效数字)

■1. 用0.618法求 $f(x)$ 的局部最优解

$$f(x) = 8x^3 - 2x^2 - 7x + 3$$

已知初始区间 $[a, b] = [0, 1]$ ，要求精度 $\varepsilon \leq 0.0001$

■2. 用Fibonacci法求 $f(x)$ 的局部最优解

$$f(x) = 8x^3 - 2x^2 - 7x + 3$$

已知初始区间 $[a, b] = [0, 1]$ ，要求精度 $\varepsilon \leq 0.0001$

2 排序算法

Sort Algorithm

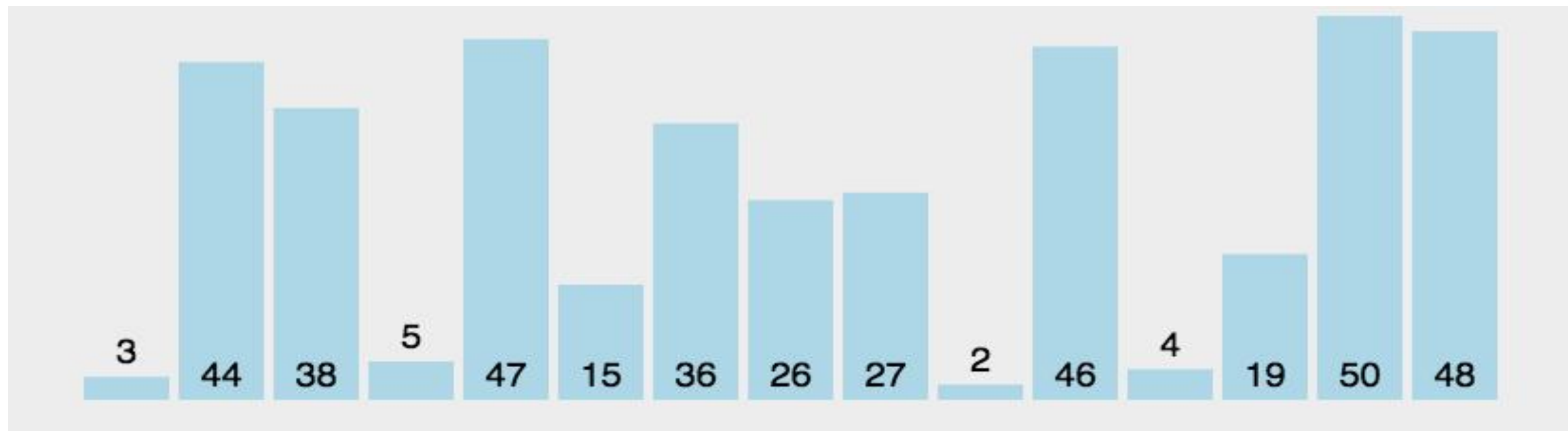
2.1 冒泡排序 (Bubble Sort)

■冒泡排序是一种简单的排序算法。它重复地走访过要排序的数列，一次比较两个元素，如果它们的顺序错误就把它们交换过来。走访数列的工作是重复地进行直到没有再需要交换，也就是说该数列已经排序完成。这个算法的名字由来是因为越小的元素会经由交换慢慢“浮”到数列的顶端。

■算法描述

- 比较相邻的元素。如果第一个比第二个大，就交换它们两个；
- 对每一对相邻元素作同样的工作，从开始第一对到结尾的最后一对，这样在最后的元素应该会是最大的数；
- 针对所有的元素重复以上的步骤，除了最后一个；
- 重复步骤1~3，直到排序完成。

■动图演示



2.2 选择排序 (Selection Sort)

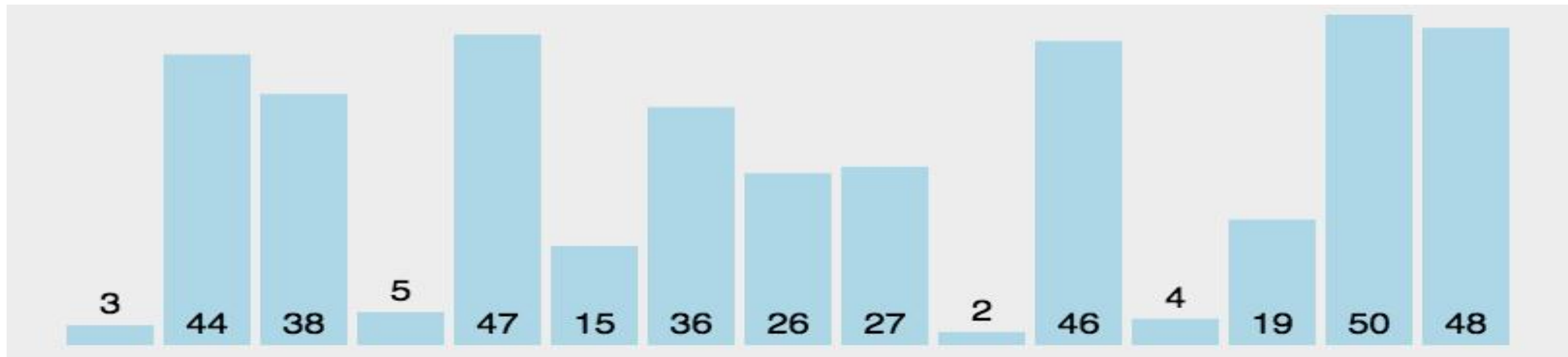
■ 选择排序(Selection-sort)是一种简单直观的排序算法。它的工作原理：首先在未排序序列中找到最小（大）元素，存放到排序序列的起始位置，然后，再从剩余未排序元素中继续寻找最小（大）元素，然后放到已排序序列的末尾。以此类推，直到所有元素均排序完毕。

■ 算法描述

■ n 个记录的直接选择排序可经过 $n-1$ 趟直接选择排序得到有序结果。具体算法描述如下：

- 初始状态：无序区为 $R[1..n]$ ，有序区为空；
- 第 i 趟排序($i=1,2,3...n-1$)开始时，当前有序区和无序区分别为 $R[1..i-1]$ 和 $R(i..n)$ 。该趟排序从当前无序区中-选出关键字最小的记录 $R[k]$ ，将它与无序区的第1个记录 R 交换，使 $R[1..i]$ 和 $R[i+1..n)$ 分别变为记录个数增加1个的新有序区和记录个数减少1个的新无序区；
- $n-1$ 趟结束，数组有序化了。

■ 动图演示



■算法分析

- 表现最稳定的排序算法之一，因为无论什么数据进去都是 $O(n^2)$ 的时间复杂度，所以用到它的时候，数据规模越小越好。唯一的好处可能就是不占用额外的内存空间了吧。理论上讲，选择排序可能也是平时排序一般人想到的最多的排序方法了吧。是对冒泡排序的一个优化

2.3 插入排序 (Insertion Sort)

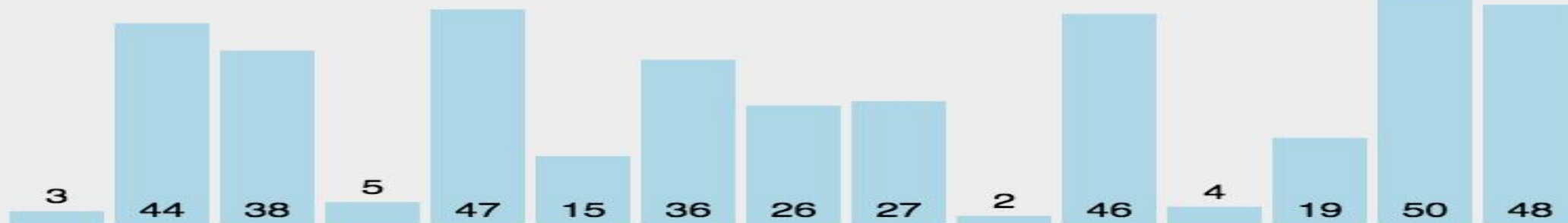
■插入排序 (Insertion-Sort) 的算法描述是一种简单直观的排序算法。它的工作原理是通过构建有序序列，对于未排序数据，在已排序序列中从后向前扫描，找到相应位置并插入。

■算法描述

一般来说，插入排序都采用 in-place 在数组上实现。具体算法描述如下：

- ✓从第一个元素开始，该元素可以认为已经被排序；
- ✓取出下一个元素，在已经排序的元素序列中从后向前扫描；
- ✓如果该元素（已排序）大于新元素，将该元素移到下一位置；
- ✓重复步骤3，直到找到已排序的元素小于或者等于新元素的位置；
- ✓将新元素插入到该位置后；
- ✓重复步骤2~5。

■动图演示



■ 算法分析

- 插入排序在实现上，通常采用in-place排序（即只需用到 $O(1)$ 的额外空间的排序），因而在从后向前扫描过程中，需要反复把已排序元素逐步向后挪位，为最新元素提供插入空间。

2.4 希尔排序 (Shell Sort)

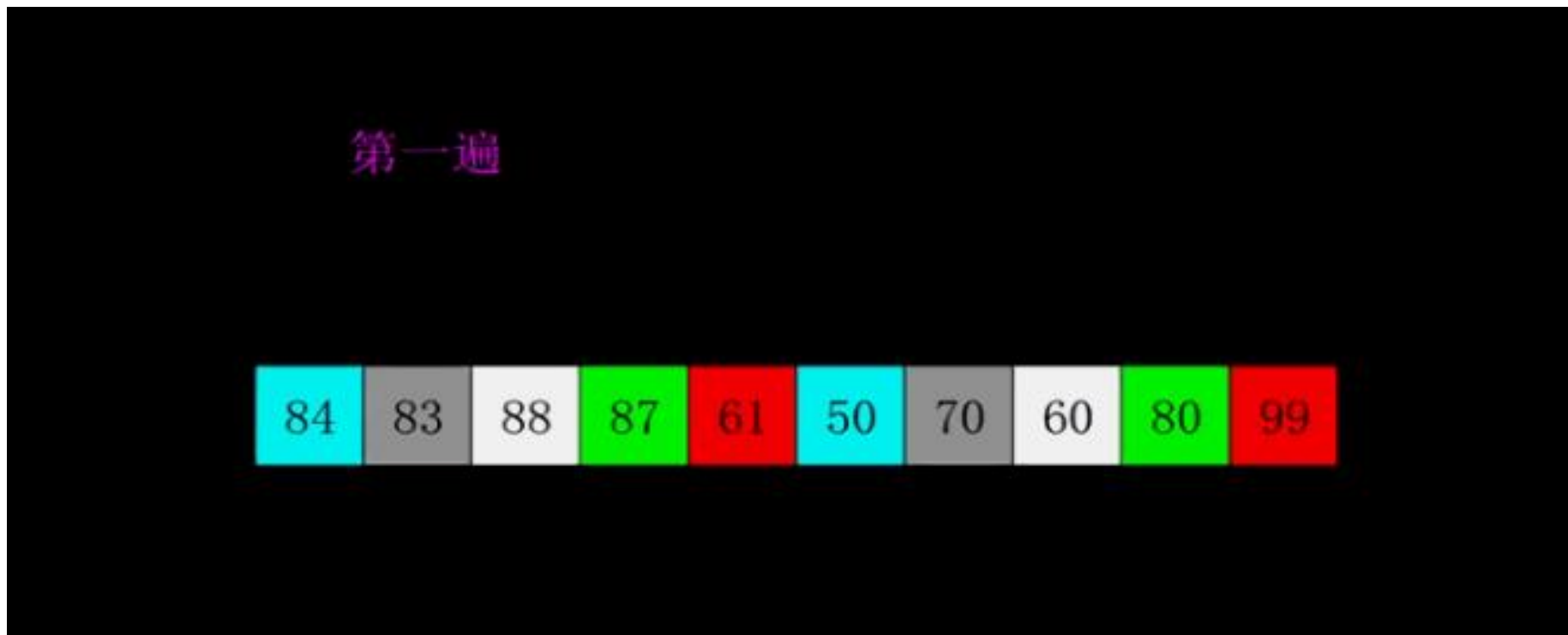
1959年Shell发明，第一个突破 $O(n^2)$ 的排序算法，是简单插入排序的改进版。它与插入排序的不同之处在于，它会优先比较距离较远的元素。希尔排序又叫**缩小增量排序**。

算法描述

先将整个待排序的记录序列分割成为若干子序列分别进行直接插入排序，具体算法描述：

- 选择一个增量序列 $t_1, t_2, \dots, t_k$ ，其中 $t_i > t_j$ ， $t_k = 1$ ；
- 按增量序列个数 k ，对序列进行 k 趟排序；
- 每趟排序，根据对应的增量 t_i ，将待排序列分割成若干长度为 m 的子序列，分别对各子表进行直接插入排序。仅增量因子为1 时，整个序列作为一个表来处理，表长度即为整个序列的长度。

动图演示



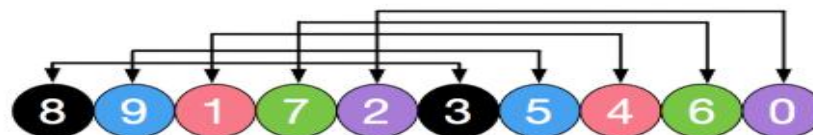
■ 算法分析

- 希尔排序的核心在于间隔序列的设定。既可以提前设定好间隔序列，也可以动态的定义间隔序列。动态定义间隔序列的算法是《算法（第4版）》的合著者Robert Sedgewick提出的。

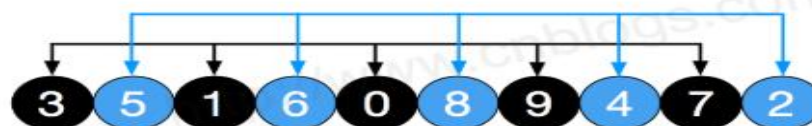
原始数组 以下数据元素颜色相同为一组



初始增量 $gap=length/2=5$ ，意味着整个数组被分为5组，[8,3] [9,5] [1,4] [7,6] [2,0]



对这5组分别进行直接插入排序，结果如下，可以看到，像3，5，6这些小元素都被调到前面了，然后缩小增量 $gap=5/2=2$ ，数组被分为2组 [3,1,0,9,7] [5,6,8,4,2]



对以上2组再分别进行直接插入排序，结果如下，可以看到，此时整个数组的有序程度更进一步啦。再缩小增量 $gap=2/2=1$ ，此时，整个数组为1组 [0,2,1,4,3,5,7,6,9,8]，如下



经过上面的“宏观调控”，整个数组的有序化程度成果喜人。

此时，仅仅需要对以上数列简单微调，无需大量移动操作即可完成整个数组的排序。



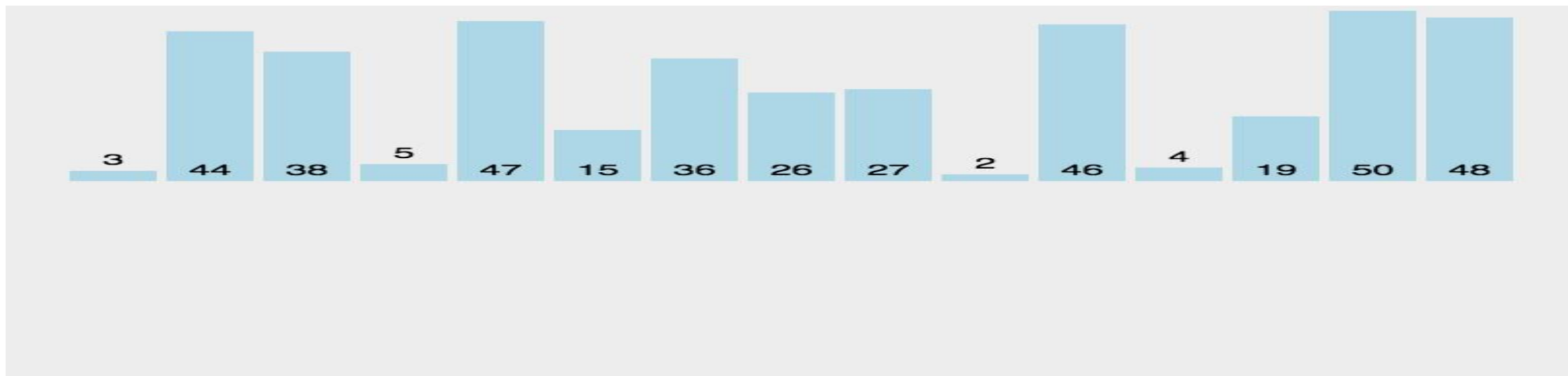
2.5 归并排序 (Merge Sort)

■归并排序是建立在归并操作上的一种有效的排序算法。该算法是采用分治法 (Divide and Conquer) 的一个非常典型的应用。将已有序的子序列合并，得到完全有序的序列；即先使每个子序列有序，再使子序列段间有序。若将两个有序表合并成一个有序表，称为2-路归并。

■算法描述

- 把长度为 n 的输入序列分成两个长度为 $n/2$ 的子序列；
- 对这两个子序列分别采用归并排序；
- 将两个排序好的子序列合并成一个最终的排序序列。

■动图演示



■算法分析

- 归并排序是一种稳定的排序方法。和选择排序一样，归并排序的性能不受输入数据的影响，但表现比选择排序好的多，因为始终都是 $O(n \log n)$ 的时间复杂度。代价是需要额外的内存空间。

2.6 快速排序 (Quick Sort)

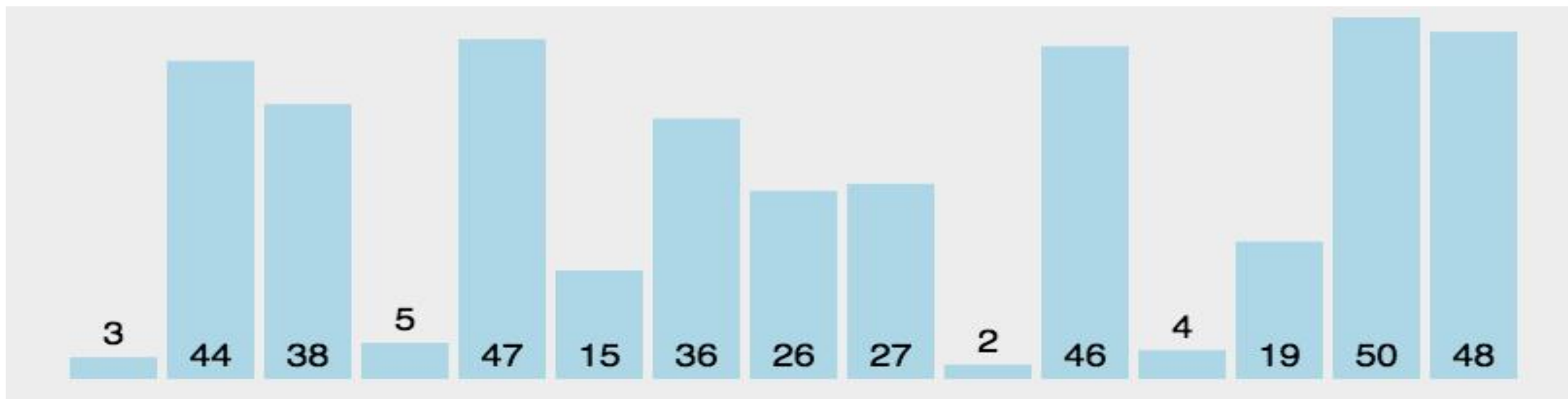
■快速排序的基本思想：通过一趟排序将待排记录分隔成独立的两部分，其中一部分记录的关键字均比另一部分的关键字小，则可分别对这两部分记录继续进行排序，以达到整个序列有序。

■算法描述

■快速排序使用分治法来把一个串 (list) 分为两个子串 (sub-lists)。
具体算法描述如下：

- 从数列中挑出一个元素，称为 “基准” (pivot)；
- 重新排序数列，所有元素比基准值小的摆放在基准前面，所有元素比基准值大的摆在基准的后面 (相同的数可以到任一边)。在这个分区退出之后，该基准就处于数列的中间位置。这个称为分区 (partition) 操作；
- 递归地 (recursive) 把小于基准值元素的子数列和大于基准值元素的子数列排序。

■动图演示



■基准是黄色，剩下比它小的标记为绿色，比它大的标记为紫色，之后插入到中间。

■设计C程序，实现以下功能：

□1. 输入待排序数列；

□2. 可进行选择排序、冒泡排序和插入排序功能。

维度	线性搜索	排序算法
目的	查找特定元素	整理数据顺序
预处理	无需	需要
适用数据	无序或少量数据	需有序或大规模数据
典型用例	单次检查、临时搜索	数据预处理、频繁查询

3 算法复杂度

Algorithm Complexity

- 对算法的评价有两个大的方面：
 - 一是对算法的维护的方便性。
 - 二是算法在实现运行时占有的机器资源的多少即算法的运行的时间和空间效率。
- 人对算法维护主要有编写、调试、改正和功能扩充等工作,这就需要在算法设计时注重算法的**可读性**。为了算法的利用率,也要考虑算法的**适用范围**,注重算法的**通用性**、**可重用性**和**可扩充性**。
- 机器对算法的运行效率主要包括时间效率和空间效率。算法在成功能的前题下最好是占用空间少而且执行时间短。
- 另外在算法实现时,要考虑算法在运行过程中与使用者进行交互的情况。这就要求,算法的交互部分要具有**友好性**和**健壮性**。

■总之，对算法的分析和评价，一般应考虑正确性、可维护性、可读性、运算量及占用存储空间等诸多因素。其中评价算法的三条主要标准是：

- (1) 算法实现所耗费的时间；
- (2) 算法实现所耗费的存储空间，其中主要考虑辅助存储空间；
- (3) 算法应易于理解，易于编码，易于调试等等。

3.1 算法的时间复杂性

■和算法**执行时间**相关的因素：

- 1) 问题中数据存储的数据结构
- 2) 算法采用的数学模型
- 3) 算法设计的策略
- 4) 问题的规模
- 5) 实现算法的程序设计语言
- 6) 编译算法产生的机器代码的质量
- 7) 计算机执行指令的速度

■算法效率的衡量方法

■通常有两种衡量算法效率的方法:

- 1) 事后统计法 (有缺点, 较少使用)
- 2) 事前分析估算法

■算法的时间效率是问题规模的函数。假如, 随着问题规模 n 的增长, 算法执行时间的增长率和 $f(n)$ 的增长率相同, 则可记作: $T(n) = O(f(n))$, 称 $T(n)$ 为算法的**渐近时间复杂度** (Asymptotic Time Complexity), 简称**时间复杂度**。O是数量级的符号。

■时间复杂度估算

因为：

算法=控制结构+原操作(固有数据类型的操作)

所以：

算法的执行时间= Σ 原操作的执行次数*原操作

语句的频度指的是该语句重复执行的次数。

一个算法转换为算法后所耗费的时间,除了与所用的计算软、硬件环境有关外,主要取决于算法中指令重复执行的次数,即语句频度相关。

一个算法中所有语句的频度之和构成了该算法的运行时间。

例如: for j:=1 to n do
 for k:=1 to n do
 x:=x+1;

语句“x:=x+1、判断k≤n?、k:=k+1”的频度是 n^2 ,
语句“j:=1、k:=1”的频度是1,
语句“判断j≤n? ; j:=j+1”的频度是n。
算法运行时间为: $3*n^2+2*n+2$ 。

对较复杂的算法计算算法的运行时间,经常从算法中选取一种对于所研究的问题来说是**基本(或者说是主要)**的原操作,以该基本操作在算法中重复执行的次数作为算法运行时间的衡量准则。这个原操作,多数情况下是最深层次循环体内的语句中的原操作。

例如:

```
for i:=1 to n do
  for j:=1 to n do
    begin
      c[i, j]=0;
      for k:=0 to n do
        c[i, j]= c[i, j]+a[i, k]*b[k, j];
      end;
```

该算法的基本操作是乘法操作,时间复杂度为 n^3

一般来说如果选用了阶乘级的算法，则当问题规模等于或者大于10时就要认真考虑**算法的适用性问题**。所以，原则上一个算法的时间复杂度，最好不要采用指数级和阶乘级的算法，而应尽可能选用多项式级或线性级等时间复杂度级别较小的算法。

对于较复杂的算法，可将它分隔成容易估算的几个部分，然后再利用"0"的求和原则得到整个算法的时间复杂度。

例：若算法的两个部分的时间复杂度分别为

$$T1(n) = O(f(n)) \quad \text{和} \quad T2(n) = O(g(n)),$$

则总的时间复杂度为：

$$T(n) = T1(n) + T2(n) = O(\max(f(n), g(n))).$$

复杂度的上界和下界是用以估计问题所需某资源的复杂程度的界限函数。如果找到解某问题的算法,其资源的复杂度为 $u(n)$, 则 $u(n)$ 是问题本身复杂度的一个**上界**。如果对任何算法,其复杂度都必需大于 $l(n)$, 则 $l(n)$ 是问题复杂度的一个**下界**。

要确定能反映出算法在各种情况下工作的数据集,选取的数据要能够反映、代表各种计算情况下的估算,包括**最好情况**下的时间复杂度(时间复杂度下界,一般记为 $T_{\max}$)、**最坏情况**下的时间复杂度(时间复杂度上界,一般记为 $T_{\min}$)、**平均情况**下的时间复杂度(一般记为 T_{avg})和有代表性的情况,通过使用这些数据配置来运行算法,以了解算法的性能。

3.2 算法的空间复杂性

■算法的存储量包括：

- 1) 输入数据所占空间；
- 2) 算法本身所占空间；
- 3) 辅助变量所占空间。

研究**算法的空间效率**，只需要分析除输入和算法之外的额外空间。若所需额外空间相对于输入数据量来说是常数，则称此算法为原地工作，否则，它应当是规模的一个函数。

算法的空间复杂度是指算法在执行过程中所占辅助存储空间的大小（也有课本定义为所占全部存储空间的大小），用 $S(n)$ 表示。与算法的时间复杂度相同，算法的空间复杂度 $S(n)$ 也可表示为：

$$S(n) = O(g(n))$$

表示随着问题规模 n 的增大，算法运行所需存储量的增长率与 $g(n)$ 的增长率相同。

3.3 算法复杂度分析

■一具体算法的时间复杂度和空间复杂度往往是不独立的，在算法设计中要在时间效率和空间效率之间折衷。

■例1：

仅依赖于问题规模的时间复杂度

有一类简单的问题，其操作具有普遍性，也就是说对所有的数据均等价地进行处理，这类算法的时间复杂度，很容易分析。

■算法1：交换i和j的内容

Temp:=i; i:=j; j:=temp;

以上三条单个语句的频度均为1，该算法段的执行时间是一个与问题规模n无关的常数。算法的时间复杂度为常数阶，记作 $T(n) = O(1)$ 。

如果算法的执行时间不随着问题规模n的增加而增长，即使算法中有上千条语句，其执行时间也不过是一个较大的常数。此类算法的时间复杂度是 $O(1)$ 。

■算法2：变量计数

- (1) $x := 0;$
- (2) for $k := 1$ to n do
- (3) $x := x + 1;$
- (4) for $i := 1$ to n do
- (5) for $j := 1$ to n do
- (6) $y := y + 1;$

■该算法段的时间复杂度为 $T(n) = O(n^2)$ 。

■当有若干个循环语句时，算法的时间复杂度是由嵌套层数最多的循环语句中最内层语句的频度 $f(n)$ 决定的。

■算法3：变量计数

- (1) $x:=1;$
- (2) for $i:=1$ to n do
- (3) for $j:=1$ to i do
- (4) for $k:=1$ to j do
- (5) $x:=x+1;$

■该算法段中频度最大的语句是(5)，从内层循环向外层分析语句(5)的执行次数：

$$\sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1 = \sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n \frac{i(i+1)}{2} = \frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2}$$

■则该算法段的时间复杂度为： $T(n) = O(n^3/12 + \text{低次项}) = O(n^3)$ 。

■ 排序算法复杂度

排序方法	时间复杂度（平均）	时间复杂度（最坏）	时间复杂度（最好）	空间复杂度	稳定性
插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
希尔排序	$O(n^{1.3})$	$O(n^2)$	$O(n)$	$O(1)$	不稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(n \log_2 n)$	$O(n^2)$	$O(n \log_2 n)$	$O(n \log_2 n)$	不稳定
归并排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n)$	稳定
计数排序	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	稳定
桶排序	$O(n+k)$	$O(n^2)$	$O(n)$	$O(n+k)$	稳定
基数排序	$O(n*k)$	$O(n*k)$	$O(n*k)$	$O(n+k)$	稳定

■某单位建立了主要零件分类目录。某次检修中需要找到物料“Intercooler Rear Backwater Gasket 中冷后回水垫片”（第4879775节点）；

■现在已经将节点ID存储在txt文件中，需要建立C程序，实现：

1. 用直接线性搜索到第4879775节点；
2. 用冒泡排序法搜索到该节点；
3. 用冒泡排序 + 0.618法搜索到该节点，并比较效率（比较次数）

节点ID	所属父类节点ID	物料名称
-1	-2	matname
880723	-1	Assembly Modules 装配组立
1717763	-1	FSES Parts
1796691	-1	临时文件夹
3371841	-1	123
4526715	-1	TW Parts
4901242	-1	Blind Flange 法兰盖
13189178	-1	NOZZLE, RECYCLING VALVE DISCHARGE 回流阀排气管
13568530	-1	Intercooler,Bundle 中间冷却器芯组
14152024	-1	Control Panel 控制盘-R200
15071388	-1	FSEP Parts
15169882	-1	CABLE, CONDUIT, AND CONTROL PIPING ARR'G'T (VFP)
652301	650969	凸面板式平焊钢制管法兰
880131	650969	英制法兰
933819	650969	非标法兰
1021148	650969	Layout Flange 配套法兰
1072983	650969	Flange 带颈平焊钢制管法兰
651451	650984	开关
651801	650984	Low Voltage Control Panel 低压控制柜
652108	650984	Low Voltage Control Panel 低压控制柜
891440	650984	Low Voltage Control Panel 低压控制柜
898988	650984	Route Voltage Panel 低压分路柜
948085	650984	Low Voltage Switch Panel 低压开关柜
993996	650984	Low Voltage Switch Panel 低压开关柜
1364335	650984	Low Voltage Switch Panel 低压开关柜
1533484	650984	Low Voltage Control Panel 低压控制柜
1533485	650984	Low Voltage Control Panel 低压控制柜
1565245	650984	Low Voltage Control Panel 低压控制柜
1565247	650984	Low Voltage Control Panel 低压控制柜



谢谢各位同学！
